



UNIVERSITÀ DEGLI STUDI

Dipartimento di Ingegneria Civile, Informatica
e delle Tecnologie Aeronautiche

Tesi di Laurea

Protecting the autonomic manager

**A comparative analysis of Quorum and
CometBFT as blockchain backends for
Self-Protecting Systems**

Corso di Laurea: Ingegneria Informatica

Candidato: Valerio Tatti

Matricola: 609824

Relatore: Prof. Stefano Iannucci

Anno Accademico: 2025/2026

Contents

1	Introduction	1
1.1	The Paradigm of Self-Protecting Systems	1
1.2	The Vulnerability of the Autonomic Manager	2
1.3	Blockchain as a Mechanism of Trust	2
1.4	Objectives and Contributions of the Thesis	3
1.5	Structure of the Thesis	4
2	Technologies Setting and Related Works	5
2.1	Autonomic Computing and the MAPE Cycle	5
2.1.1	The Four Aspects of Self-Management	6
2.1.2	The Structure of Autonomic Elements	7
2.1.3	Security Challenges	7
2.2	Blockchain in Permissioned Environments	8
2.2.1	Smart Contracts and Response Logic	8
2.2.2	Characteristics of Permissioned Environments	8
2.2.3	Alignment with SPS Requirements	9
2.3	Distributed Consensus Algorithms	9
2.3.1	Proof-of-X: What Can Untrusted Environments Teach Us?	10
2.3.2	Crash Fault Tolerance	10
2.3.3	Leader-Based Algorithms	11
2.3.4	Byzantine Fault Tolerance	12
2.4	Autonomic Manager Security's Current State	13
2.4.1	Traditional Hardening and Centralized Security	13
2.4.2	Trusted Execution Environments (TEEs)	13
2.4.3	Distributed Ledger Technologies	14
3	Self-Protecting Systems Architecture	15
3.1	Design Principles and Rationale	15
3.2	Architectural Decomposition and Trust Boundaries	16
3.2.1	Managed System	16
3.2.2	Manager System	17
3.2.3	Isolation and Containerization Requirements	17

3.2.4	Actuator Blindness and Unidirectional Control	18
3.3	System's State Formalization	18
3.3.1	Observation Sets and Redundancy	18
3.3.2	Proposal and Consolidation Semantics	19
3.3.3	Replicated Decision State	19
3.3.4	Determinism and Auditability	20
3.3.5	State-Action Map Limitations and Temporal Context	20
3.4	End-to-End Protection Workflow	21
3.4.1	Managed Monitoring and Attack Detection	21
3.4.2	Alert and Transaction Submission	21
3.4.3	Consensus-Guided State Transition	21
3.4.4	Response Actuation and Feedback Loop	22
3.5	Correctness and Actuation Properties	22
3.5.1	Safety Invariants	22
3.5.2	Liveness Guarantees	23
3.6	Failure, Recovery and Security	23
3.6.1	Crash and Restart of Non-Validator Components	23
3.6.2	Validator Failure and Quorum Loss	24
3.6.3	Network Partitions	24
3.6.4	Threat Model	24
3.6.5	Adversarial Strategies	25
3.6.6	Voting and Validation Differences	25
3.6.7	Residual Risk	26
4	Requirement Alignment: Quorum vs. CometBFT	27
4.1	Specific Requirements for SPS Applications	27
4.2	Analysis of GoQuorum	27
4.3	Analysis of CometBFT	27
4.4	Architectural Comparison	27
5	Implementation details	28
5.1	Experimental Setup and Network Topology	28
5.2	Rust vs Solidity middleware	28
5.3	Alert detection and response loop	28
5.4	Blockchain config generation	28
6	Experiments and results	29
6.1	Evaluation Methodology	29
6.2	Performance Analysis	29
6.2.1	Effectiveness of the Deduplication Mechanism	29
6.2.2	End-to-End Latency under Low Traffic	29

6.2.3	Transaction Throughput under Increasing Load	29
7	Conclusions and Future Work	30
7.1	Summary of Research Findings	30
7.2	Limitations of the Study	30
7.3	Future Directions	30

List of Figures

List of Tables

Introduction

In modern IT infrastructures, the rapid shift toward cloud-native, containerized microservices has dramatically expanded the digital attack surface. The escalation of sophisticated, high-velocity cyber threats and the ever-growing volume of sensitive data housed in these environments have driven the need for computing systems capable of defending themselves autonomously. Within this context, the autonomic computing paradigm has become a cornerstone of modern security engineering, aiming to reduce human operational burden and accelerate threat response. However, while automation resolves many latency and coordination challenges, it introduces new systemic vulnerabilities. If the automated security logic itself is targeted, it can become a vector for devastating privilege escalation and system-wide compromise.

1.1 The Paradigm of Self-Protecting Systems

Self-Protecting Systems [1] (SPS) are autonomic software systems designed to detect, analyze, and mitigate security threats dynamically with limited or no human intervention. Following the classic autonomic feedback loop, an SPS typically comprises two primary macro-components: a *managed system* and an *autonomic manager*. The managed system represents the production environment, services, or software applications exposed to potential attacks. Conversely, the autonomic manager serves as the central controller responsible for executing the security loop.

To achieve continuous protection, the autonomic manager organizes its operations into a pipeline governed by two main functional sub-components:

- **Intrusion Detection Systems (IDS):** Passive or active security sensors distributed across the managed system to monitor traffic, logs, or system calls and detect anomalous behavior or known attack signatures.
- **Intrusion Response Systems (IRS):** Decision-making engines responsible for selecting and executing the appropriate response actions (countermeasures)

to neutralize or mitigate the detected threats.

1.2 The Vulnerability of the Autonomic Manager

Although highly effective in mitigating threats directed at the managed system, this canonical design suffers from a critical structural vulnerability: the centralization of the autonomic manager. An adversary who penetrates the infrastructure can target the autonomic manager itself. By compromising this central controller, the attacker can silence intrusion alerts, disable response mechanisms, or even manipulate the manager into executing malicious countermeasures (e.g., disconnecting legitimate network segments or isolating critical database services).

Indeed, protecting the controller that protects the system remains an open and central challenge in autonomic security research. If the security logic is centralized and compromised, the integrity of the entire infrastructure is undermined, rendering the self-protecting features not only ineffective, but actively weaponized against the system.

1.3 Blockchain as a Mechanism of Trust

To eliminate the single point of failure represented by a centralized manager, our research [2] has proposed distributing the autonomic manager across a permissioned blockchain network. In this decentralized architecture, blockchain technology acts as a tamper-proof ledger and a distributed state machine (DSM) that logs security events and executes response logic securely through smart contracts.

This distributed approach establishes fundamental security guarantees that collectively safeguard the autonomic loop. To prevent a single compromised IDS sensor from injecting false alerts and triggering unauthorized countermeasures, the smart contract enforces a consensual voting mechanism. Under this scheme, updates to the shared security state are only committed once they receive corroborating reports from multiple independent, heterogeneous IDSs, ensuring that isolated sensor compromises do not lead to malicious actuations.

Also, by deploying the autonomic manager across a network of validator nodes running a Byzantine Fault-Tolerant (BFT) consensus protocol, the security loop gains strong structural resilience. At blockchain level, this mechanism tolerates the active compromise of up to $1/3$ of validators while preserving ledger safety and finality under partial network compromise. This guarantee is distinct from IDS-side voting, which is the mechanism used to validate the semantic correctness of observations before they affect the consolidated monitored state.

Generally, the system is designed to reduce the attack surface as much as possible treating actuators as black boxes (blind), the actuators receive signed mitigation commands directly from the blockchain state, rather than querying the state of potentially compromised application containers directly.

1.4 Objectives and Contributions of the Thesis

While the theoretical integration of blockchain into the SPS architecture provides outstanding security guarantees, the practical choice of the underlying blockchain technology is not neutral. A blockchain-based autonomic manager has unique requirements: it must handle sparse, highly event-driven alert traffic with exceptionally low latency and high determinism. Conversely, many features native to public blockchains (such as virtual-machine-level gas metering, token economics, transaction fees, and dynamic peer discovery) add unnecessary complexity, expand the trusted computing base (TCB), and increase the attack surface.

The natural baseline choice for permissioned enterprise setups is GoQuorum (a fork of Ethereum). However, this thesis argues that general-purpose smart contract platforms incur severe, non-intrinsic performance overheads due to interpreted virtual machine (EVM) bytecode execution, rigid transaction fee mechanisms, and fixed-interval block production. As an alternative, this work explores CometBFT (the successor of Tendermint), a lower-level BFT consensus engine that supports highly optimized, application-specific distributed state machines.

The primary objective of this thesis is to present a comprehensive qualitative and empirical comparison between a conventional general-purpose platform (GoQuorum) and a requirement-driven, application-specific alternative (CometBFT) to determine which architecture best supports the performance and cyber-resilience demands of modern Self-Protecting Systems.

Specifically, the core contributions of this thesis are as follows:

1. **Requirement Formulation:** We define and classify a comprehensive set of functional and non-functional requirements (required, desirable, and not needed) for blockchain technology in the specific context of an autonomic SPS loop.
2. **Architectural Analysis:** We conduct a thorough qualitative comparison of GoQuorum and CometBFT, highlighting the architectural mismatch of general-purpose virtual machines versus application-specific state machines.
3. **Prototype Implementation:** We design and implement a realistic, fully containerized SPS prototype emulated via Kathará on top of Docker. The

prototype integrates three heterogeneous IDS engines (Snort, Suricata, and Zeek) protecting a vulnerable web application (OWASP Juice Shop).

4. **Deduplication Middleware:** We propose and develop a custom deduplication middleware layer for the member nodes. This layer filters repetitive alerts, preventing mempool saturation on the ledger keeping transaction number upper limited per block.
5. **Empirical Evaluation:** We perform extensive automated benchmarks under controlled attack conditions, comparing GoQuorum and CometBFT across key performance indicators, including end-to-end response latency, transaction throughput under heavy load, and CPU resource utilization.

1.5 Structure of the Thesis

To thoroughly explore the theoretical background, architectural design, and empirical evaluation of the proposed systems, the remainder of this thesis is structured as follows:

Chapter 2 – Technologies Setting and Related Work: Introduces the foundational concepts of autonomic computing, the MAPE cycle, Byzantine fault-tolerant consensus, the structural differences between public and permissioned blockchain systems, and reviews the state of the art in autonomic security.

Chapter 3 – A Cyber-Resilient Architectural Model: Details the technology-agnostic architecture of the Self-Protecting System, focusing on the interactions between IDSs, the smart contract state, and the blind actuators.

Chapter 4 – Requirement Alignment: Quorum vs. CometBFT: Formally identifies the blockchain properties strictly required for SPS applications and theoretically compares Quorum’s general-purpose execution environment with CometBFT’s application-specific design.

Chapter 5 – Implementation Details: Describes the prototype implementation, the containerized network topology emulated via Kathará, and the custom middleware developed for the evaluation.

Chapter 6 – Experiments and Results: Analyzes the performance of the two blockchain backends under varying workloads, presenting metrics on end-to-end latency, transaction throughput, and resource consumption.

Chapter 7 – Conclusions and Future Work: Summarizes the findings of the comparative analysis, acknowledges current limitations, and outlines directions for future research in decentralized system protection.

Technologies Setting and Related Works

Before diving into the specific idea and implementation behind the system described in this research, it is necessary to introduce the basic theoretical concepts that inspired this architecture. These concepts will later provide us with the tools to comprehend by which metrics a blockchain-based autonomic agent should be measured, which paradigm first described autonomic systems, which security issues we can expect, how the current state-of-the-art (SOTA) solutions address them, and how our architecture compares to them.

2.1 Autonomic Computing and the MAPE Cycle

The paradigm of Autonomic Computing formally emerged in October 2001, when IBM released a manifesto identifying an upcoming software complexity crisis as the primary obstacle to further progress in the IT industry [3]. As computing environments began to weigh in at millions of lines of code, the expertise required for installation, configuration, and maintenance approached a limit on human capability to train new professionals. This challenge was exacerbated by the necessity of integrating several heterogeneous environments into corporate-wide systems and extending them globally across the Internet. IBM observed that relying solely on innovations in programming methods would not suffice to manage such massive and complex systems, which often require timely responses to rapid streams of changing demands.

To address this problem, IBM proposed the vision of autonomic computing: systems capable of managing themselves based on high-level objectives from human administrators [3]. The term "autonomic" was chosen for its biological connotation, tracing an analogy to the human autonomic nervous system. Just as the biological system governs vital, low-level functions such as heart rate and body temperature without conscious effort, autonomic computing aims to free human administrators from the burden of managing low-level system operations through

automation.

2.1.1 The Four Aspects of Self-Management

The essence of autonomic computing is self-management, intended to provide software that can adjust its behaviour in accordance with human objectives without their intervention [3]. IBM characterizes self-management through four fundamental aspects, often referred to as the "self-*" properties. Those principles have since materialized into concrete industry standards and architectural paradigms that govern contemporary network infrastructures:

- **Self-configuration:** Systems must automatically configure components and integrate themselves into complex environments following high-level policies. New components should be able to incorporate seamlessly, treating other components as black boxes. This modular perspective prefigured modern containerization paradigms, where container runtimes such as Docker have become the industry standard for application deployment.
- **Self-optimization:** Autonomic systems continually seek opportunities to improve their own performance and efficiency. In environments like complex middleware or databases that possess hundreds of manually set parameters, the system must monitor, experiment with, and tune these parameters dynamically. A notable industrial implementation of this concept is cloud-based auto-scaling (e.g., Amazon Web Services Auto Scaling), which dynamically adjusts resource provisioning to match fluctuating performance demands.
- **Self-healing:** The system is designed to automatically detect, diagnose, and repair localized software and hardware problems. Using knowledge of the system configuration and logs, the autonomic manager can identify root causes of failures, match them against known patches, and retest the system without manual debugging. In contemporary orchestration systems, self-healing is commonly implemented via automated health checks and liveness probes (e.g., in Kubernetes), which automatically restart or replace failing service instances.
- **Self-protection:** Autonomic systems defend the infrastructure as a whole against malicious attacks [4]. They use sensor reports to anticipate problems and take proactive steps to avoid or mitigate systemwide failures. An example of such proactive defense is represented by modern Web Application Firewalls (WAFs), such as Cloudflare's edge security systems, which autonomously detect, analyze, and block malicious traffic patterns.

2.1.2 The Structure of Autonomic Elements

Architecturally, autonomic systems are composed of elements that manage their own internal behavior and relationships with others in accordance with established policies [3]. An autonomic element typically consists of one or more managed elements coupled with a single autonomic manager.

- **Managed Element:** This component can be found in non-autonomic systems and can be a hardware resource (such as storage or a CPU) or a software resource (such as a database or a web service).
- **Autonomic Manager:** The manager distinguishes the autonomic element by monitoring the managed element and its environment, and executing actions based on the analysis of their information.

The internal logic of the autonomic manager is structured around a fundamental control loop known as the **MAPE-K** cycle [3, 5]:

1. **Monitor:** Collects, aggregates, and filters information from the managed element and its environment.
2. **Analyze:** Matches a diagnosis on the system's high-level status based on monitored information and triggers a response.
3. **Plan:** Creates or selects a sequence of actions that matches the organization's objectives.
4. **Execute:** Applies the derived plan / action to the managed element through effectors.
5. **Knowledge:** A central repository for policies, historical data, and system configurations that supports the other four phases defining the organization's objectives.

2.1.3 Security Challenges

Autonomic elements, unlike typical Web services, do not honor requests unconditionally; they provide services only if doing so is consistent with their internal goals [3].

This high degree of autonomy introduces critical system-level issues that cannot be ignored when evaluating attack surfaces. Because these systems rely on high-level goals, they are vulnerable to new forms of attack where an adversary might manipulate policies to gain systemic leverage [6]. This highlights the necessity for a secure-by-design management infrastructure that can address the "Who watches the watchers?" dilemma of autonomic elements [6, 7].

2.2 Blockchain in Permissioned Environments

The integration of blockchain technology to register system state represents an innovative approach to ensuring cyber-resilience [2]. However, in this specific application context, the choice of the underlying blockchain platform is not neutral, and it is necessary to define which blockchain properties are beneficial for a Self-Protecting System. A fundamental distinction must be made between public (permissionless) blockchains and private (permissioned) blockchains [8].

2.2.1 Smart Contracts and Response Logic

Before analyzing network access models, it is essential to define the mechanism through which the blockchain enforces security policies: the *smart contract*. A smart contract is a self-executing distributed software deployed on a blockchain that automatically enforces predefined rules and agreements between nodes without the need for intermediaries [9].

Once deployed, smart contracts run deterministically across all nodes in the network, ensuring the transparency, trustlessness, and immutability of the encoded logic [9]. In the context of a Self-Protecting System (SPS), the integrity of the smart contract's state and its execution is guaranteed by the inherent properties of the blockchain itself [2]. This enables the autonomic manager to maintain a consistent, system-wide view and coordinate responses even under partial compromise [2].

2.2.2 Characteristics of Permissioned Environments

Public blockchains are designed as open networks where anyone can participate. In contrast, a permissioned environment requires nodes to authenticate each other, often within a fixed trust boundary [8]. This architectural difference fundamentally alters the threat model and the system's performance requirements.

In a permissioned network, many attacks traditionally critical for permissionless blockchains, such as Eclipse attack, are naturally mitigated because participant identities are controlled and membership is typically static [2].

Since permissionless environments adopt a fundamentally "trustless" model, they must employ strict, and often resource-expensive, consensus algorithms. In a permissioned blockchain, consensus can be simplified because all participating nodes are pre-vetted, identified, and operate within a predefined deployment. Consequently, the economic dynamics, gas fees, token incentives, and game-theory mechanisms typical of public cryptocurrencies designed to coordinate anonymous actors become functionally obsolete [10]. Furthermore, general-purpose virtual machines (VMs) such as the Ethereum Virtual Machine (EVM) often introduce unnecessary complexity, processing overhead, and a broader attack

surface when the system only requires a constrained, deterministic runtime for state transitions [10].

Instead, the focus in such environments shifts towards a specific set of essential properties. As highlighted in recent literature evaluating blockchain implementations for Self-Protecting Systems [10], the required features are:

- **Fault-Tolerant consensus with strong finality:** To tolerate a bounded number of compromised nodes and commit updates deterministically.
- **Static permissioned membership:** To match the fixed trust boundary of the deployment.
- **Cryptographic transaction signing:** To ensure the authenticity and accountability of alerts submitted by IDS's associated member nodes.

Additionally, desirable properties include in-memory replicated state handling (as the shared state in an SPS is typically small), low and predictable latency to guarantee timely countermeasures, and push-based notifications to immediately trigger actuators upon state updates [10].

2.2.3 Alignment with SPS Requirements

For an SPS, the blockchain's sole functions are:

- To gather a shared state from Intrusion Detection Systems (IDS) [2].
- To ensure the reliable functioning of the response logic even in the presence of security faults in the managed system or in the autonomic manager [2].
- To communicate the response logic's decisions to the actuators securely [2].

Under these assumptions, permissioned environments offer indispensable advantages, such as static membership that matches the trust boundaries of a real-world deployment [10]. Furthermore, cryptographic transaction signing ensures the authenticity and accountability of sensor submissions [2, 10]. Finally, while blockchain provides an immutable audit trail for forensics, permissioned settings may allow for configurable data-retention policies, which is particularly relevant for resource-constrained or embedded systems [10].

Ultimately, the relevant blockchain properties in this domain are not openness or tokenization, but minimality, determinism, and resilience [10].

2.3 Distributed Consensus Algorithms

Blockchain technologies, despite their significant advantages, introduce substantial technological and implementation challenges that must be addressed to render

them practically useful. Consequently, every architectural decision must be evaluated carefully. Because a blockchain is fundamentally a distributed system, it requires a robust protocol to maintain network communication and ensure the reliability of the network's collective decisions. Distributed consensus algorithms address exactly this fundamental question: what is the protocol by which information is validated and permanently recorded as "official" within the distributed ledger? The selection of a consensus algorithm is by no means context-agnostic; on the contrary, aligning the consensus mechanism with the operational threat model is the primary driver of our architectural choices for the Self-Protecting System (SPS).

2.3.1 Proof-of-X: What Can Untrusted Environments Teach Us?

As previously discussed in Section 2.2, permissionless consensus mechanisms such as Proof-of-Work (PoW) and Proof-of-Stake (PoS) are unsuitable for our specific deployment due to their reliance on tokenomics, high latency, and massive resource consumption. However, these "Proof-of-X" algorithms serve as a fundamental theoretical inspiration.

They were explicitly designed to reach consensus in fully untrusted environments where any participating node could be malicious or anonymous. Their success provides a critical lesson for our SPS architecture: in environments susceptible to targeted cyberattacks and partial compromises, we cannot rely on consensus protocols that implicitly and excessively trust the other participating nodes. If a compromised autonomic manager node were implicitly trusted, it could easily broadcast false states or suppress legitimate IDS alerts, completely neutralizing the system's defensive capabilities. Therefore, the chosen algorithm must fundamentally account for and tolerate malicious actors.

2.3.2 Crash Fault Tolerance

Crash Fault Tolerance (CFT) algorithms are built for systems where computers might suddenly crash, lose power, or drop off the network. The core idea behind all CFT algorithms is a strong assumption of honesty. They expect that while a node might fail silently, it will never act maliciously or lie. As long as a majority of the nodes are still running and following the rules, the system can stay consistent. The problem is that this built-in trust makes CFT algorithms incredibly fragile in untrusted environments. If an attacker takes over a node and starts feeding the network bad data on purpose, the entire system can easily be subverted.

- **Paxos:** Introduced by Leslie Lamport, Paxos achieves consensus through a strictly mathematical multi-phase process: *Prepare* (a proposer selects

a sequence number and requests promises from acceptors), *Accept* (upon receiving a majority of promises, the proposer broadcasts the value), and *Learn* (the agreed value is propagated). Its technical complexity arises from managing concurrent proposals and overlapping majorities without a stable leader.

- **Raft:** Designed specifically as a more comprehensible alternative to Paxos, Raft achieves CFT by decomposing the consensus problem into three distinct mechanisms: leader election, log replication, and safety. A strongly elected leader continuously sends *AppendEntries* heartbeats to followers. If a follower's randomized timeout expires without receiving a heartbeat, it transitions to a candidate state to trigger a new election.

2.3.3 Leader-Based Algorithms

Leader-based algorithms try to make things simpler and much faster by putting one single node in charge. This elected coordinator, usually called the leader or primary, takes on the responsibility of organizing transactions, making the final decisions, and telling all the other follower nodes what the new state is. Because everything goes through one point, these systems can process data very efficiently. However, this design creates an obvious weak spot. If the leader crashes, the whole system stops until the followers can run an election and agree on a replacement. Just like general CFT algorithms, traditional leader-based approaches also assume that all nodes are honest, which means they are generally not equipped to handle nodes that try to sabotage the network.

- **Viewstamped Replication:** A state replication protocol that relies on a primary node to sequence requests within a specific configuration period called a "view". The primary assigns monotonic sequence numbers and multicasts operations to backups. If the primary fails, the replicas initiate a view change protocol, electing a new primary via a deterministic round-robin schedule (based on the view number modulo the number of nodes) and synchronizing log entries before resuming.
- **Bully Algorithm:** A classic leader election algorithm where participating nodes are assigned unique, strictly hierarchical identifiers. When a node detects a failure, it sends an *ELECTION* message to all nodes with higher IDs. If no node replies, it assumes leadership and broadcasts a *COORDINATOR* message. If a higher-ID node replies, the lower-ID node yields. Crucially, if a crashed node with a high ID recovers, it immediately preempts the current leader.

2.3.4 Byzantine Fault Tolerance

Byzantine Fault Tolerance (BFT) algorithms solve the biggest problem with CFT by preparing for the worst. They are designed to keep the network running correctly even if a specific portion of the nodes fails completely, goes offline, or is actively hijacked by an attacker trying to spread conflicting information. For a network of $3f + 1$ nodes, up to f nodes can turn fully malicious without breaking the system. It gives us the strong security we need to survive compromised nodes, while still being fast and efficient enough for a permissioned environment without needing the heavy and wasteful mechanics of "Proof-of-X".

To guarantee that malicious nodes cannot trick the network into creating a fork, BFT algorithms rely on a strict multi-phase voting process. While different protocols use slightly different names, the core structure is the same. First is the **Propose** (or *Pre-prepare*) phase, where a designated leader broadcasts a new block. Then comes the **Prevote** (or *Prepare*) phase, where all other nodes verify the block and broadcast a vote to confirm they received a valid proposal. Finally, the **Precommit** (or *Commit*) phase begins once a node sees that a supermajority of the network has prevoted. In this final step, nodes broadcast their final approval. Only when a supermajority of precommit votes is collected does the block get permanently written to the ledger. This double-checking mechanism ensures that no honest node commits a block unless it is absolutely certain the majority of the network is doing the same.

- **QBFT (Quorum Byzantine Fault Tolerance):** A consensus algorithm designed specifically for permissioned, Ethereum-based networks like Quorum. It operates through this deterministic, three-step voting process (*Pre-prepare*, *Prepare*, *Commit*) driven by a proposer selected via a round-robin schedule. A block requires $2f + 1$ cryptographic validation signatures from a predefined pool of $3f + 1$ validators to be committed, yielding immediate transaction finality without the possibility of forks.
- **Tendermint/CometBFT:** The consensus engine underlying CometBFT provides highly efficient BFT state machine replication. It enforces a strict separation between the application logic and the consensus layer via the Application Blockchain Interface (ABCI). Tendermint utilizes a continuous gossiping protocol and requires a two-thirds supermajority of voting power at both the *prevote* and *precommit* stages. Unlike traditional periodic block production, its event-driven model generates blocks only when transactions are pending.

2.4 Autonomic Manager Security's Current State

As previously discussed, the question of "Who watches the watchers?" remains a central dilemma in the field of autonomic computing. If an adversary successfully compromises the autonomic manager, they can blind the system to ongoing attacks or even weaponize its response mechanisms against the underlying infrastructure. Over the years, various architectural and technological paradigms have been proposed to secure the manager, each presenting distinct advantages and critical limitations.

2.4.1 Traditional Hardening and Centralized Security

The most conventional approach to protecting the manager relies on defense-in-depth strategies deployed within a centralized architecture. This typically involves isolating the autonomic manager within a highly secure environment, fortified by strict network firewalls, stringent access controls, and dedicated monitoring infrastructure. While straightforward to implement, this approach inherently relies on securing a single point of failure rather than providing intrinsic architectural resilience. If an adversary exploits a zero-day vulnerability in the manager's dependencies or breaches the surrounding network perimeter, the system's entire defensive capability is instantly compromised. Consequently, while system hardening remains a mandatory prerequisite for any component interfacing with untrusted actors, it is insufficient as a standalone security paradigm.

2.4.2 Trusted Execution Environments (TEEs)

To fundamentally reduce the attack surface, researchers have explored hardware-assisted isolation using Trusted Execution Environments (TEEs), such as Intel Software Guard Extensions (SGX) that imposed itself as the current state-of-the-art in the field. In this model, the autonomic manager's critical logic and sensitive data are executed within a secure hardware enclave. Even in the event of a complete compromise of the host operating system or hypervisor, the adversary theoretically cannot read or tamper with the manager's execution. This is achieved through transparent memory encryption and strict privilege separation; the TEE operates at a highly privileged hardware level, ensuring that untrusted software running in higher-level protection rings cannot access the enclave's memory space.

Despite these strong theoretical guarantees, TEEs exhibit critical practical limitations. Operating on shared physical hardware exposes the enclave to a broad spectrum of microarchitectural side-channel attacks. High-profile vulnerabilities such as Spectre and Meltdown exploit speculative execution and CPU caching mechanisms to leak enclave secrets. Software patching mitigations rarely eliminate

the entire attack surface, as evidenced by the subsequent emergence of sophisticated attacks like Microarchitectural Data Sampling (MDS, better known as ZombieLoad). Finally, relying on TEEs introduces severe vendor lock-in and fails to resolve the single-point-of-failure problem; an adversary can simply disconnect or crash the physical machine hosting the enclave, resulting in a complete Denial of Service (DoS).

2.4.3 Distributed Ledger Technologies

As established in Section 2.2, integrating permissioned blockchain technology directly into the autonomic manager provides a robust architectural solution. By replicating the manager's state across multiple independent nodes and encoding the response logic within deterministic smart contracts, the system fundamentally eliminates the centralized weak spot while simultaneously maintaining an immutable, cryptographically verifiable audit trail of all security events.

Crucially, implementing this distributed architecture requires a rigorous protocol to manage node coordination in an environment susceptible to malicious actors. As analyzed in Section 2.3, Byzantine Fault Tolerance (BFT) algorithms provide the exact mathematical guarantees needed for this task. By requiring a supermajority agreement for every state transition, the distributed manager can securely tolerate partial network compromise without halting operations or accepting tampered data. This synergy of blockchain architecture and BFT consensus can resolve the foundational dilemma of autonomic computing with a structural solution, redefining the manager from a vulnerable isolated entity into a resilient, decentralized and physically separate network.

Self-Protecting Systems Architecture

This chapter details the architectural core of the proposed blockchain-based Self-Protecting System (SPS), focusing on how to address the centralized autonomic manager’s problem explored in 2.4. The model presented herein is grounded in the foundational decentralized architecture introduced by [2] and subsequently refined through the requirement-driven analysis by our research [10]. The primary objective is to formalize the system decomposition, state model, end-to-end protection workflow, threat assumptions, and resilience properties. These components justify the integration of a permissioned, Byzantine Fault Tolerant (BFT) distributed ledger as the secure decision substrate for the autonomic manager.

3.1 Design Principles and Rationale

Traditional Self-Protecting Systems are typically designed around a centralized MAPE-K (Monitor-Analyze-Plan-Execute over Shared Knowledge) feedback loop [3]. While functionally effective, this centralized model introduces a fundamental security asymmetry: the managed system can be replicated, segmented, and protected, whereas the autonomic manager itself remains a high-value, centralized target. If an adversary successfully compromises the manager, they can suppress valid security alerts, trigger unauthorized response actions, or stall mitigation entirely. This effectively neutralizes the self-protection mechanism and grants the attacker complete control over the production environment [6, 7].

To address this structural vulnerability, the architecture proposed in this thesis distributes the decision authority of the autonomic manager across replicated, isolated nodes coordinated through a permissioned blockchain using BFT consensus [2]. The core design choice is not simply the adoption of blockchain as a modern storage medium, but its use as a replicated state machine where:

- **Consensus-driven state updates:** Monitored variables representing the managed system state are updated only after achieving quorum-based agreement among independent observers,

- **Deterministic transitions:** Response decisions are modeled as deterministic functions of the globally agreed state, guaranteeing that honest nodes reach identical conclusions,
- **Auditable actuation:** Actuation instructions are pushed from a tamper-evident, globally replicated log, ensuring source accountability and post-incident forensic auditability.

By encoding the autonomic manager within a Byzantine Fault Tolerant ledger, trust is shifted from a single, hardening-dependent process to a formal fault model. The central security question shifts to determining the number of components that can fail or be corrupted before the safety and liveness of the control loop are violated, exposing the managed system to threats.

At the architectural level, this model is deliberately technology-agnostic. It does not dictate a specific Intrusion Detection System (IDS) engine, response planner, or blockchain implementation. Instead, it defines a strict security contract among the components: authenticated observations, deterministic state transitions, replicated BFT consensus, and auditable, blind actuation. A crucial aspect of this design is the requirement for *immediate finality* in the underlying consensus protocol. Unlike public permissionless blockchains that rely on probabilistic finality (e.g., waiting for multiple block confirmations), a software control loop cannot act on decisions that might be subsequently rolled back. A block commit must represent a final, immutable transition to avoid executing conflicting or unauthorized actions.

3.2 Architectural Decomposition and Trust Boundaries

To ensure structural resilience, the architecture decomposes the system into distinct functional planes, separated by explicit trust boundaries and network isolation. The system maintains the classic autonomic split between the *managed system* (the production environment) and the *manager system* (the security controller) [3, 2], but introduces strict boundaries to limit the lateral propagation of attacks.

3.2.1 Managed System

The managed system represents the production workload to be protected (e.g., containerized application services, microservices, or database nodes). From a security perspective, this domain is assumed to be untrusted, as it is directly exposed to external users and potential attackers. The behavior and operational

health of the managed system are captured through a set of monitored variables:

$$\mathbf{V} = \{v_1, v_2, \dots, v_k\}.$$

These variables represent resource-level, network-level, or application-level signals (such as CPU saturation, database query anomalies, authentication failure rates, or protocol misuse indicators). At any time instant t , the observable state of the managed system is represented by the vector:

$$s_t = \langle v_1(t), v_2(t), \dots, v_k(t) \rangle.$$

Security-relevant events (e.g., a Denial of Service attack or a SQL injection attempt) modify one or more coordinates of s_t . The architecture does not couple itself to any specific detection paradigm, allowing the use of signature-based, anomaly-based, or hybrid detection systems to monitor these variables.

3.2.2 Manager System

The manager system is the actuation plane in charge of processing state observations and executing responses. It is decomposed into three logical planes:

1. **Detection Plane:** Composed of multiple independent IDS instances that observe overlapping portions of the managed system's state variables $\mathbf{V}$.
2. **Consensus-and-Decision Plane:** A permissioned blockchain network that maintains the replicated decision state and executes the response selection logic.
3. **Actuation Plane:** A set of response executors (actuators) that receive committed mitigation commands and enforce them on the managed system.

Each active component in the detection and actuation planes is paired with a blockchain-facing node. In our design, we distinguish between *validating nodes* (which participate in the BFT consensus protocol) and *member or light nodes* (which submit transactions or listen to events without validating blocks). This separation decouples functional roles from consensus overhead, allowing the validator set to remain small and efficient while the number of sensors and actuators scales.

3.2.3 Isolation and Containerization Requirements

The security of the manager system relies heavily on containerization and network isolation [2]. Each IDS, actuator, and blockchain node must run in its own isolated container, deployed across physical or virtual hosts. In our prototype, this environment is emulated using the Kathará framework [11] on top of Docker.

Under this model, the container boundary serves as the primary security perimeter. If an attacker compromises an application service in the managed system, they cannot pivot into the manager system’s containers unless an explicit, authenticated communication path exists. The attack surface of the manager system is restricted to three channels:

- The telemetry and input channels of the IDSs,
- The transaction submission APIs exposed by the blockchain nodes,
- The control paths through which actuators apply countermeasures.

3.2.4 Actuator Blindness and Unidirectional Control

To prevent attackers from manipulating responses, actuators are modeled as *blind* [2]. Actuators do not query the managed system directly to inspect its state, nor do they accept instructions from it. Instead, they operate exclusively as passive consumers of the committed decisions stored in the blockchain.

If an actuator were allowed to pull state variables directly from a compromised application container to confirm an attack, the attacker could exploit this interactive channel to inject malicious payloads or trigger unauthorized actions. By enforcing actuator blindness, the control loop establishes a strict unidirectional data flow: IDSs push observations to the consensus plane, the blockchain validates and commits the state transition, and the resulting response is pushed to the actuators. This design pattern ensures that an actuator only executes commands that have passed through BFT consensus.

3.3 System’s State Formalization

To prevent ambiguity and support formal verification, the control pipeline of the decentralized manager is formalized as a deterministic state machine.

3.3.1 Observation Sets and Redundancy

Let $\mathcal{D} = \{d_1, d_2, \dots, d_m\}$ be the set of deployed IDSs. Each IDS d_i is configured to monitor a specific subset of state variables, denoted by the observation scope $O_i \subseteq \mathbf{V}$. For any monitored variable $v_j \in \mathbf{V}$, we define its observer set $\mathcal{D}_{v_j}$ as the subset of IDSs that monitor it:

$$\mathcal{D}_{v_j} = \{d_i \in \mathcal{D} \mid v_j \in O_i\}.$$

To ensure semantic robustness, the architecture requires $|\mathcal{D}_{v_j}| \geq 3$ for all critical variables, utilizing heterogeneous detection technologies (e.g., combining Snort,

Suricata, and Zeek). Using diverse detection engines ensures that a zero-day exploit or signature bypass targeting one specific IDS implementation will not compromise the consolidated observation, as the remaining heterogeneous observers can still report the event accurately.

3.3.2 Proposal and Consolidation Semantics

When an IDS d_i detects a change in a variable v_j , it submits a state proposal transaction to the blockchain:

$$p = \langle id_{ids}, v_j, \hat{x}, t \rangle$$

where id_{ids} is the unique identifier of the submitting IDS (d_i), v_j is the target variable, $\hat{x}$ is the proposed value or classification, and t is the detection timestamp.

Inside the smart contract or the application state machine, incoming proposals are collected within a sliding aggregation window. To filter out noise or a compromised sensor's false reports, the state is updated only when a quorum predicate is satisfied. In its simplest majority form, the consolidation predicate is defined as:

$$count_{v_j}(\hat{x}) \geq q_j, \quad q_j = \left\lceil \frac{|\mathcal{D}_{v_j}|}{2} \right\rceil + 1.$$

In stricter Byzantine configurations with $3f_d + 1$ observers, the quorum threshold can be adjusted to $2f_d + 1$ to tolerate up to f_d Byzantine (malicious or failed) IDSs.

3.3.3 Replicated Decision State

The global state of the decentralized manager is formalized as:

$$\Sigma_t = \langle S_t, R_t, I_t \rangle$$

where:

- S_t is the consolidated monitored state (represented as a parameter-status map),
- R_t is the response relation (represented as a state-action map mapping consolidated states to countermeasures),
- I_t is the internal control metadata (such as active response flags, execution status, and cooldown timers).

Every committed transaction deterministically transforms the manager state: $\Sigma_t \xrightarrow{T_i} \Sigma_{t+1}$. To represent this state in a concrete implementation (such as the

prototypes analyzed in Chapters 4 and 5), the consolidated monitored state S_t is modeled as a bit-vector (a sequence of boolean flags) of length N , where N represents the number of distinct attacks or anomalies detectable by the system. The response relation R_t is implemented as a map associating each state vector with a specific mitigation action. For example, in our prototype, R_t maps state vectors to command strings or script identifiers that the actuators are authorized to execute.

3.3.4 Determinism and Auditability

Deterministic execution is a fundamental requirement for the manager state machine. Given the same initial state Σ_0 and an identical sequence of transactions $T_1, T_2, \dots, T_n$, all non-faulty validator nodes must compute the exact same sequence of states $\Sigma_1, \dots, \Sigma_n$ and produce the exact same outputs. This provides two key security benefits:

- **Consistency:** It prevents split-brain scenarios, ensuring that all replicas agree on which response action to trigger,
- **Forensics:** The entire history of committed transitions is recorded in an immutable ledger, allowing post-incident reconstruction and verification of security policies.

To preserve determinism across the system, both the state transition function (executed by the consensus nodes) and the alert generation process (performed by the detection components) must adhere to these constraints. For instance, any runtime logic or actuator selection mechanism must avoid relying on non-deterministic variables (such as local clocks or system randomness) so that all replicas compute identical state updates.

3.3.5 State-Action Map Limitations and Temporal Context

A direct state-action map assumes a Markov-like property, meaning that the current consolidated state S_t contains all the information necessary to select a response. While this simplifies the design and guarantees deterministic replay, it limits the manager's ability to detect multi-stage or low-and-slow attacks that unfold over time.

To overcome this limitation, the state representation must incorporate historical context directly into the state variables. This can be achieved by storing memory variables (such as event counters, sliding-window registers, and cooldown flags) within the internal metadata I_t . Rather than relying on external logs, the state machine itself tracks temporal patterns, ensuring that the inputs to the response logic remain self-contained and deterministic.

3.4 End-to-End Protection Workflow

The proposed architecture establishes a closed-loop control pipeline that processes security events from detection to mitigation. The workflow consists of four sequential, causally ordered stages:

3.4.1 Managed Monitoring and Attack Detection

When a malicious or anomalous event occurs in the managed system, it alters one or more monitored variables. The independent IDSs observing these variables detect the anomaly and evaluate the event type. Because different IDSs utilize different detection engines (e.g., signature-based vs. protocol analysis) and run on distinct containers, they may flag the attack at slightly different times, introducing a temporal skew. The architecture accommodates this asynchrony by aggregating alerts incrementally on the blockchain's smart contract.

3.4.2 Alert and Transaction Submission

The detecting IDS sends an alert to an associated Member Node that wraps the observation into a proposal, signs it cryptographically, and submits it as a transaction to its paired blockchain node. The entire permissioned blockchain network receives the transaction and verifies the signature against the membership service. Because the blockchain nodes are isolated from the untrusted managed system, an external attacker who has compromised an application container cannot easily inject forged transactions or manipulate the consensus membership without compromising multiple IDSs. This process is discussed in more detail in Section 3.6.6.

3.4.3 Consensus-Guided State Transition

The validating nodes receive the verified transactions, order them using the BFT consensus protocol, and package them into blocks. Once a block is committed, the transitions are executed by the smart contract (or the ABCI application). If the aggregated proposals satisfy the voting predicate, the consolidated state S_t is updated, and the response logic determines the appropriate countermeasure a_t . This BFT-guided commitment guarantees that the response decision is final and agreed upon by a qualified majority of the network.

3.4.4 Response Actuation and Feedback Loop

Once a response a_t is committed, the blockchain node notifies the selected actuator using a push-based subscription channel (e.g., Ethereum event logs or CometBFT Websocket events). The blind actuator receives the instruction and executes the mitigation action (e.g., blocking an IP address, updating firewall rules, or restarting a container) via a secure channel such as SSH.

The enforcement of the countermeasure alters the managed system's state, returning the monitored variables $\mathbf{V}$ to their normal, secure range. The IDSs observe this return to a safe state and submit proposals to clear the active threat state. The state machine then updates, clears the active response flag, and terminates the response execution, closing the feedback loop.

3.5 Correctness and Actuation Properties

To evaluate the reliability of the actuation plane, we define formal correctness properties that must be preserved by the architecture.

3.5.1 Safety Invariants

Safety in the actuation plane requires that the manager never triggers unauthorized or conflicting actions. We express this through two formal invariants:

1. **No Unauthorized Actuation:** Let A be the set of mitigation actions, and let $E_{exec}(a)$ denote the event that action $a \in A$ is executed by an actuator. Let $\Sigma_{commit}(a)$ denote the event that the replicated state machine commits a transition emitting the response action a . The safety invariant requires:

$$E_{exec}(a) \implies \Sigma_{commit}(a)$$

This is guaranteed by actuator blindness and push-based communication channels.

2. **Agreement (No Conflicting Decisions):** For any block height h , and any two non-faulty consensus nodes i and j , their committed states must be identical:

$$\Sigma_h^i = \Sigma_h^j$$

This is guaranteed at the blockchain layer by BFT consensus protocols, but must be properly addressed in the smart contract implementation for IDS-level voting.

The combination of these two invariants is critical. Reaching consensus without actuator discipline (blindness) could still allow unauthorized local execution, while actuator discipline without consensus could result in divergent control histories across different nodes.

3.5.2 Liveness Guarantees

Liveness ensures that the system reacts to threats in a timely manner. Formally, if a security breach changes the monitored variables v_j at time t such that the response logic requires action a , then a non-faulty actuator must execute a at some time $t' > t$, assuming that the number of Byzantine failures does not exceed the consensus protocol's threshold.

Liveness is practically constrained by validator availability, transaction submission latency, and block production times. The block production model of the blockchain backend is a relevant parameter when choosing the technology to implement the system and will be addressed in Chapter. //TODO aggiungere al capitolo su comparazione!! Under a fixed-interval block production model (such as in GoQuorum), transactions must wait in the mempool for the next block timer, introducing a systematic delay. In contrast, an event-driven block production model (such as in CometBFT) creates blocks as soon as transactions are available, minimizing reaction latency under sparse threat conditions.

3.6 Failure, Recovery and Security

A resilient SPS must maintain predictable behavior not only during active cyber-attacks but also during system faults, partitions, and recovery events.

3.6.1 Crash and Restart of Non-Validator Components

If an IDS container crashes, the system's detection coverage temporarily decreases, but the consensus and decision plane continues to operate. Upon restart, the IDS re-authenticates and resumes submitting proposals without requiring a system-wide reconfiguration thanks to the self-healing properties of containerized applications.

If an actuator container crashes and restarts, it must not replay stale mitigation actions. To prevent this, the actuator queries the internal metadata I_t on the blockchain. If the metadata indicates that the action associated with the current state has already been successfully executed, the actuator suppresses execution, preventing duplicate actuation cycles that could disrupt the managed system.

3.6.2 Validator Failure and Quorum Loss

Let N_v be the total number of validator nodes in the permissioned network. In a standard BFT consensus protocol, the quorum size Q_v required to commit a block is:

$$Q_v = \left\lfloor \frac{2N_v}{3} \right\rfloor + 1.$$

The system can tolerate up to $f_v = \lfloor \frac{N_v-1}{3} \rfloor$ Byzantine (malicious or crashed) validators.

If the number of failed validators remains within this threshold ($f \leq f_v$), the network continues to commit transactions and guarantees safety and liveness. If the number of failures exceeds f_v , the consensus protocol stalls (safety-preserving stall). While the manager ceases to process new state updates, it preserves its last committed state, avoiding split-brain or divergent state transitions.

3.6.3 Network Partitions

In the event of a network partition, the CAP theorem applies //TODO Citare il CAP theorem, forse scriverlo in breve da qualche parte, da valutare. The BFT-based manager prioritizes consistency (safety) over availability (liveness). Only the partition containing a supermajority of validators (Q_v) can continue committing state transitions. Minority partitions are unable to reach consensus and stall. While this temporarily reduces responsiveness in isolated segments, it prevents the creation of divergent control histories, preserving the integrity of the global state.

3.6.4 Threat Model

The threat model assumes an active adversary with network access to the managed service, whose objective is to subvert the autonomic manager. A successful attack on the manager is defined by two failure modes:

1. **Omission:** An attack occurs and is detected, but the manager fails to execute the required response,
2. **Illegitimate Actuation:** An actuator executes a countermeasure that was not authorized by the consolidated manager state.

The threat model is defined under the following assumptions:

- The container isolation primitive is robust, preventing container escapes,
- Network segments connecting the manager (IDSs and Blockchain) containers are isolated from public traffic,

- The only direct, untrusted inputs entering the manager are the exposed variables monitored by the IDSs (e.g., the network traffic),
- The smart contract or ABCI state machine code is correctly and securely implemented, and
- Node identities are verified and governed by a permissioned membership model.

3.6.5 Adversarial Strategies

An attacker attempting to subvert the manager can pursue three main strategies:

1. **Input Poisoning:** The attacker attempts to compromise multiple IDSs to inject false observations, forcing the manager into an incorrect state.
2. **Consensus Subversion:** The attacker attempts to compromise more than $1/3$ of the validator nodes to take control of block commitment and rewrite history.
3. **Actuator Corruption:** The attacker attempts to compromise the actuators directly to block mitigations or trigger unauthorized actions.

The architecture raises the cost of each strategy:

- Input poisoning is mitigated by the multi-IDS voting logic and the use of heterogeneous detection engines that prevent technology-specific faults,
- Consensus subversion is prevented by BFT consensus, which requires compromising a supermajority of validators, and
- Actuator corruption is mitigated by actuator blindness, which prevents direct communication from compromised application containers.

3.6.6 Voting and Validation Differences

This section formalizes the security guarantees of the architecture, showing how the remaining attack paths are restricted and highly resource-intensive for an adversary. Let N_v be the number of validators in the consensus layer. The system can tolerate up to f_v Byzantine validators, where:

$$f_v = \left\lfloor \frac{N_v - 1}{3} \right\rfloor.$$

Let N_d be the number of independent IDSs monitoring a state variable v_j . If majority voting is applied, the observation layer can tolerate up to f_d compromised IDSs, where:

$$f_d = \left\lfloor \frac{N_d - 1}{2} \right\rfloor.$$

To successfully subvert the manager, an attacker must compromise either a majority of the IDSs observing a target variable ($> f_d$) or a supermajority of the consensus validators ($> f_v$). This two-layer threshold design prevents the single point of failure inherent in centralized autonomic managers.

Redundancy without diversity is vulnerable to common-mode failures. If all IDSs run the exact same software, a single vulnerability could allow an attacker to compromise the entire set simultaneously. The architecture therefore recommends using heterogeneous IDS engines (e.g., Snort, Suricata, and Zeek). This increases the attacker's costs by requiring them to develop and deploy multiple distinct exploits, making the possibility of compromising the autonomic manager highly improbable since the architecture can increase the number of diverse technologies at will.

3.6.7 Residual Risk

The architecture does not protect against host-level compromises or vulnerabilities in the underlying container runtime. If an attacker gains root access to the physical host, they can bypass container boundaries and compromise the nodes. Additionally, bugs in the smart contract or ABCI code remain a critical vulnerability, as a replicated state machine will faithfully execute and replicate flawed logic. In any computer system, expanding the trusted computing base is a choice that must be treated with caution, therefore, secure coding and formal verification of the response transition function remain essential.

Requirement Alignment: Quorum vs. CometBFT

4.1 Specific Requirements for SPS Applications

4.2 Analysis of GoQuorum

4.3 Analysis of CometBFT

4.4 Architectural Comparison

Implementation details

5.1 Experimental Setup and Network Topology

5.2 Rust vs Solidity middleware

5.3 Alert detection and response loop

5.4 Blockchain config generation

Experiments and results

6.1 Evaluation Methodology

6.2 Performance Analysis

6.2.1 Effectiveness of the Deduplication Mechanism

6.2.2 End-to-End Latency under Low Traffic

6.2.3 Transaction Throughput under Increasing Load

Conclusions and Future Work

7.1 Summary of Research Findings

7.2 Limitations of the Study

7.3 Future Directions

Bibliography

- [1] Eric Yuan, Naeem Esfahani, and Sam Malek. “A Systematic Survey of Self-Protecting Software Systems”. In: *ACM Trans. Auton. Adapt. Syst.* 8.4 (Jan. 2014). ISSN: 1556-4665. DOI: 10.1145/2555611. URL: <https://doi.org/10.1145/2555611>.
- [2] Tommaso Caiazzzi et al. “A Novel Architecture for Cyber-Resilient Self-Protecting Systems Based on Blockchain”. In: *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. 2025, pp. 1–8. DOI: 10.1109/COMPSAC65507.2025.00010.
- [3] Jeffrey O Kephart and David M Chess. “The vision of autonomic computing”. In: *Computer* 36.1 (2003), pp. 41–50.
- [4] Eric Yuan, Naeem Esfahani, and Sam Malek. “A systematic survey of self-protecting software systems”. In: *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 8.4 (2014), pp. 1–41.
- [5] Paolo Arcaini, Elvinia Riccobene, and Patrizia Scandurra. “Modeling and Analyzing MAPE-K Feedback Loops for Self-Adaptation”. In: *2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 2015, pp. 13–23. DOI: 10.1109/SEAMS.2015.10.
- [6] David M Chess, Charles C Palmer, and Steve R White. “Security in an autonomic computing environment”. In: *IBM Systems journal* 42.1 (2003), pp. 107–118.
- [7] Irdin Pekaric et al. “A systematic review on security and safety of self-adaptive systems”. In: *Journal of Systems and Software* 203 (2023), p. 111716.
- [8] Xin Wang et al. “Bft in blockchains: From protocols to use cases”. In: *ACM Computing Surveys (CSUR)* 54.10s (2022), pp. 1–37.
- [9] Vitalik Buterin et al. “Ethereum white paper”. In: *GitHub repository* 1 (2013), pp. 22–23.
- [10] Simone Alberio et al. “Which Blockchain Technology Best Supports Self-Protecting Systems? A comparison between Quorum and CometBFT”. In: *Proceedings of the 8th Distributed Ledger Technology Workshop (DLT 2026)*. In press. Pula, Italy: CEUR-WS.org, June 2026.

- [11] M. Scazzariello, L. Ariemma, and T. Caiazzzi. “Kathará: A Lightweight Network Emulation System”. In: *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*. 2020.